

Práctica: Sistemas Numéricos y Aritmética Binaria

Materia: Arquitectura de Computadoras — Ingeniería de Software **Entorno:** Ubuntu Linux (para las partes de script/verificación) **Formato de entrega:** Documento `.md` en el repositorio del curso, con las respuestas y (donde se indique) el script usado para verificar.

Nivel 1 — Conversión de bases (fundamentos)

1.1 Convierte los siguientes números decimales a binario: 45, 128, 255, 1000

1.2 Convierte los siguientes números binarios a decimal: 1011, 10110110, 11111111, 100000000

1.3 Convierte a hexadecimal los siguientes números binarios: 10101100, 111100001111, 1010101010101010

1.4 Convierte de octal a decimal: 17, 254, 777

Nivel 2 — Aritmética binaria básica

2.1 Realiza las siguientes sumas binarias (muestra el acarreo): 1011 + 0110 11101 + 01011

2.2 Realiza las siguientes restas binarias usando préstamo (borrow): 1100 - 0101 10000 - 00111

2.3 Multiplica en binario: 1011 × 0101

Nivel 3 — Complemento a dos

3.1 Representa los siguientes números decimales en complemento a dos usando 8 bits: -45 , -1 , -128 , 27

3.2 Dado el número en complemento a dos de 8 bits 11010110 , indica su valor decimal.

3.3 Realiza la resta $45 - 60$ usando complemento a dos con registros de 8 bits, y verifica el resultado.

Nivel 4 — Desbordamiento (overflow) y banderas

4.1 Determina si ocurre overflow al sumar, en complemento a dos de 8 bits: $01111111 + 00000001$ $10000000 + 11111111$

4.2 Explica, con tus propias palabras, la diferencia entre *carry* (acarreo) y *overflow* en una suma con signo, y da un ejemplo de cada caso usando registros de 4 bits.

4.3 (práctico en Ubuntu) Escribe un programa en C que sume dos enteros `int8_t` propensos a overflow (por ejemplo $120 + 20$), compílalo con `gcc`, ejecútalo, y explica por qué el resultado impreso no es el esperado matemáticamente. Adjunta el comando de compilación y la salida.

Nivel 5 — Punto flotante (IEEE 754)

5.1 Representa el número decimal 10.25 en formato IEEE 754 de precisión simple (32 bits), mostrando signo, exponente y mantisa por separado.

5.2 Dado el patrón de bits IEEE 754 de 32 bits $1\ 10000010\ 011000000000000000000000$, calcula su valor decimal.

5.3 Explica qué es la pérdida de precisión en punto flotante y demuéstalo con un ejemplo numérico (por ejemplo, sumar $0.1 + 0.2$ en punto flotante).

Nivel 6 — Integración: script de verificación (el más difícil)

6.1 (práctico en Ubuntu) Escribe un script en Python (o Bash) que:

- Reciba un número entero decimal.
- Imprima su representación en binario, octal y hexadecimal.
- Imprima su representación en complemento a dos de 8, 16 y 32 bits (si aplica).
- Indique si el número, al representarse en 8 bits con signo, produce overflow.

Prueba el script con al menos 5 valores distintos, incluyendo un caso límite (por ejemplo 127, 128, -128, -129, 0), y adjunta las salidas obtenidas.

6.2 A partir del resultado de tu script, redacta un párrafo explicando qué pasaría si un sensor de hardware reporta temperaturas como enteros de 8 bits con signo y se produce un overflow silencioso: ¿qué valor vería el sistema y por qué es peligroso en un contexto real (por ejemplo, un sistema embebido)?